

事前準備：ローカル永続化やること サイト2 設計文書

作成日：2024年12月19日

演習作成マニュアル：6.1 事前準備に基づく設計

1. テーマと学習目標の明確化

演習テーマ

「ローカル永続化やることサイト2」 - JavaScriptとlocalStorage APIを使用したタスク管理アプリの永続化機能実装

最終学習目標

JavaScriptでのlocalStorage操作を理解し、タスク管理アプリにデータ永続化機能を実装できるようになる

具体的な習得目標

- localStorage API の基本操作（setItem, getItem, removeItem, clear）の理解と実践
- JSON変換（stringify/parse）を用いたオブジェクトデータの保存・復元
- エラーハンドリング を含む安全なlocalStorage操作の実装
- ブラウザストレージ の概念と適切な使い分けの理解

対象者設定（演習作成マニュアル1.3準拠）

- プログラミング経験：0～3ヶ月程度
- 前提知識：
 - HTML/CSS基礎知識
 - JavaScript基本構文（変数、関数、DOM操作）
 - 「やること動的サイト」レベルの実装経験
- localStorage経験：初体験

2. 全体のストーリー設計

学習の流れ（段階的習得）

ベースアプリ → localStorage基本体験 → 自動保存 → 復元 → 完了状態保存 → 削除連携

各ステップの積み上げ設計

- ベース**：動作するタスク管理アプリ（localStorage機能なし）
- 体験**：ブラウザ開発者ツールでlocalStorageを直接触って理解
- 実装**：段階的にlocalStorage機能を追加
- 完成**：完全に永続化されたタスク管理アプリ

認知的負荷の管理

- 1ステップ1〜2概念**：localStorage基本操作、JSON変換、エラーハンドリングを段階的に習得
- 実践重視**：各ステップで必ず動作確認を行い、成果を体感
- 漸進的複雑化**：単純な保存から複雑なデータ構造まで段階的に

3. 各ステップの詳細分解

ステップ1: localStorage基本操作の体験（15分）

学習目標：localStorage APIの基本操作を理解する

主要概念：setItem, getItem, removeItem, clear

体験内容：ブラウザ開発者ツールでのinteractive学習

成果物：localStorage操作の基本理解

ステップ2: タスク追加時の自動保存機能（20分）

学習目標：タスク追加時にlocalStorageへ自動保存できる

主要概念：JSON.stringify(), try-catchエラーハンドリング

実装内容：addTask()関数にlocalStorage保存機能を追加

成果物：タスク追加と同時に永続化されるアプリ

ステップ3: ページ読み込み時のタスク復元機能（20分）

学習目標：保存したタスクを起動時に復元できる

主要概念：JSON.parse(), null値処理, DOMContentLoaded

実装内容：初期化時にlocalStorageからタスクを読み込み

成果物：ブラウザを閉じてても再開するアプリ

ステップ4: タスク完了状態の保存と復元（15分）

学習目標 : タスクの完了状態も永続化できる

主要概念 : オブジェクトプロパティの保存, 状態同期

実装内容 : toggleTask()関数にlocalStorage更新機能を追加

成果物 : 完了状態も記憶するアプリ

ステップ5: タスク削除時のlocalStorageからの削除（10分）

学習目標 : タスク削除がlocalStorageにも反映される

主要概念 : データ整合性, CRUD操作の完結

実装内容 : deleteTask()関数にlocalStorage更新機能を追加

成果物 : 完全に永続化対応したタスク管理アプリ

4. 補足資料のテーマ選定

4段階詳細度設計（演習作成マニュアル4章準拠）

テーマ1: localStorage API詳解

- **基本(1分)** : setItem, getItem, removeItem, clearの最低限解説
- **詳細(3分)** : API仕様、制限事項、よくある間違いと注意点
- **深掘り(コラム)** : sessionStorageとの違い、Cookieとの比較、IndexedDB紹介
- **実践応用** : 複数アプリ間でのデータ共有、Storage イベントの活用

テーマ2: JSON操作（stringify/parseの仕組み）

- **基本(1分)** : JSON.stringify(), JSON.parse()の基本的な使い方
- **詳細(3分)** : 変換できないデータ型、循環参照、replacer/reviver関数
- **深掘り(コラム)** : JSONフォーマットの仕様、他の序列化手法との比較
- **実践応用** : 複雑なオブジェクト構造の保存、バージョン管理対応

テーマ3: データ構造設計（Webアプリでのデータ管理）

- **基本(1分)** : タスクオブジェクトの設計思想（id, text, completed）
- **詳細(3分)** : 正規化、一意ID生成、データ整合性
- **深掘り(コラム)** : 関係データベース vs NoSQL、フロントエンド状態管理
- **実践応用** : 大規模アプリでのデータ設計パターン、パフォーマンス考慮

テーマ4: ブラウザストレージ比較

- **基本(1分)** : localStorage vs sessionStorage vs Cookie
- **詳細(3分)** : 容量制限、永続性、スコープ、セキュリティ
- **深掘り(コラム)** : IndexedDB, WebSQL (非推奨)、Service Worker Cache
- **実践応用** : 用途別ストレージ選択指針、ハイブリッド活用法

5. 演習用ベースコードの準備

技術仕様

- **HTML** : セマンティックマークアップ、アクセシビリティ対応
- **CSS** : モダンCSS、レスポンシブデザイン、CSS Grid/Flexbox活用
- **JavaScript** : ES6+記法、モジュラー設計、コメント充実

機能仕様 (localStorage機能なし)

- ☒ タスクの追加 (Add Task)
- ☒ タスクの削除 (Delete Task)
- ☒ タスクの完了切り替え (Toggle Complete)
- ☒ 進捗バー自動更新 (Progress Bar)
- ☒ 完了メッセージ表示 (Completion Message)
- ☒ **データ永続化 (localStorage)** → 演習で段階的に実装

コード品質基準

- **教育的コード** : 学習者が理解しやすい変数名、関数名
- **段階的拡張対応** : localStorage機能を追加しやすい構造
- **エラー処理** : 基本的なバリデーション実装済み
- **デバッグ支援** : console.log活用でデバッグしやすい設計

6. メイン演習と補足資料の連携設計

自然な誘導パターン

> **より詳しく知りたい場合:** 「補足資料1: localStorage API詳解」で、より深い仕様や他のストレージとの比較を段階

的に学べます。

情報の補完的役割分担

- **メイン演習**：実践と概要、すぐに手を動かせる
- **補足資料**：詳細と体系化、必要な時に必要な深さで学べる

重複・矛盾の防止

- **用語統一**：全資料でlocalStorage APIの表記を統一
- **コード整合性**：サンプルコードの変数名、関数名を統一
- **難易度調整**：メイン → 補足の自然な学習流れを確保

7. 品質担保方針

演習作成マニュアル6.4品質チェックリスト準拠

- ☒ **指示の明確性**：具体的なキー名、値、期待動作を明記
- ☒ **穴埋めの適切性**：学習目標と直結、適切な難易度
- ☒ **解答との整合性**：穴埋めから論理的に導出可能
- ☒ **別解の考慮**：想定される別解をフォロー
- ☒ **メイン演習と補足資料の連携**：自然で補完的
- ☒ **技術的正確性**：コードと説明の誤りを徹底排除

セルフレビュー観点

1. **学習者視点**：初学者が詰まりそうなポイントの事前対策
2. **指導者視点**：教えやすい資料構成、エラー対応方法の整備
3. **技術的視点**：最新のWeb標準に準拠、将来性のある内容

8. 成功指標

学習者の達成目標

- ☐ localStorage APIの基本操作を説明できる
- ☐ タスクアプリに永続化機能を自力で実装できる

- ☐ JSON変換の仕組みを理解している
- ☐ ブラウザストレージの使い分けを判断できる

教材の品質指標

- ☐ ステップ通りに進めて100%動作するアプリが完成する
- ☐ 各ステップの学習時間が想定範囲（10-20分）内
- ☐ 補足資料が自然に活用される
- ☐ 学習者が「次も学びたい」と感じる

この設計文書に基づき、演習作成マニュアル6.2「各ステップの作成手順」に従って高品質な演習資料を作成していきます。